Lecture 7. Higher-Order Functions and Algebraic Data Types

Programming Paradigms, Fall 2026 — 7 Oct 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are
2. Functions as Values
3. Folds
4. Types of Our Own
5. **`Maybe`** and **`Either`**
6. Type Classes

Where We Are

This Lecture

Last week: equations, patterns, recursion on lists, and a type of our own.

This Lecture

Last week: equations, patterns, recursion on lists, and a type of our own.

This week: functions as values, and types with several constructors.

This Lecture

Last week: equations, patterns, recursion on lists, and a type of our own.

This week: functions as values, and types with several constructors.

Four things to get out of it:

1. **Reading** the type of a function that takes or returns a function.
2. **Reading** a type with type variables: what it allows, and what it rules out.
3. **Rewriting** a recursion with an accumulator as a fold.
4. **Designing** a type for the domain, and reading the `=>` in a signature.

This Lecture



Last week: equations, patterns, recursion on lists, and a type of our own.

This week: functions as values, and types with several constructors.

Four things to get out of it:

1. **Reading** the type of a function that takes or returns a function.
2. **Reading** a type with type variables: what it allows, and what it rules out.
3. **Rewriting** a recursion with an accumulator as a fold.
4. **Designing** a type for the domain, and reading the `=>` in a signature.

Last week ended with four types that said too little.
Today each of them gets a better one.

Quiz 6: `type` Against `newtype`

Many of you asked why an alias lets a price through and a wrapper does not.

Quiz 6: `type` Against `newtype`

Many of you asked why an alias lets a price through and a wrapper does not.

a new type

```haskell
newtype Grams = MkGrams Double
```

```
ghci>  :type MkGrams
MkGrams ::  Double -> Grams
```

`MkGrams` is a function, and the only way to make a `Grams`.

Quiz 6: `type` Against `newtype`

Many of you asked why an alias lets a price through and a wrapper does not.

a new type

```
newtype Grams = MkGrams Double
```

```
ghci> :type MkGrams
MkGrams :: Double -> Grams
```

two new names

```
type Grams  = Double
type Rubles = Double
```

`MkGrams` is a function, and the only way to make a `Grams`.

GHC replaces an alias by its definition before it checks the types.
Thus, for GHC, `postage` has the type `Double -> Double`.

Quiz 6: `type` Against `newtype`

Many of you asked why an alias lets a price through and a wrapper does not.

a new type

```
newtype Grams = MkGrams Double
```

```
ghci> :type MkGrams
MkGrams :: Double -> Grams
```

two new names

```
type Grams  = Double
type Rubles = Double
```

`MkGrams` is a function, and the only way to make a `Grams`.

GHC replaces an alias by its definition before it checks the types.
Thus, for GHC, `postage` has the type `Double -> Double`.

In practice, the constructor usually gets the name of the type: `Grams`.
A type name appears only in types, and a constructor only in expressions.

Functions as Values

A Function Takes a Function



```haskell
totalBy :: (Double -> Double) -> [Double] -> Double
totalBy f ws = sum (map f ws)
```

A Function Takes a Function



```haskell
totalBy :: (Double -> Double) -> [Double] -> Double
totalBy f ws = sum (map f ws)
```

The parentheses make the function `Double -> Double` a single argument.

A Function Takes a Function



```haskell
totalBy :: (Double -> Double) -> [Double] -> Double
totalBy f ws = sum (map f ws)
```

The parentheses make the function `Double -> Double` a single argument.

```
ghci>  totalBy postage weights
2200.0
ghci>  totalBy (* 2) weights
15480.0
```

A Function Takes a Function



```haskell
totalBy :: (Double -> Double) -> [Double] -> Double
totalBy f ws = sum (map f ws)
```

The parentheses make the function `Double -> Double` a single argument.

```
ghci>  totalBy postage weights
2200.0
ghci>  totalBy (* 2) weights
15480.0
```

A **higher-order function** takes a function as an argument, or returns one.
We have used two of them already, in Racket and in Haskell: `map` and `filter`.

A Function Without a Name



We want the prices of several tariffs, with insurance for a parcel worth 3000:

```haskell
insuredAll :: [Double] -> [Double]
insuredAll prices = map withInsurance prices
  where
    withInsurance p = insured 3000 p
```

The local function is used once, and its name says little.

A Function Without a Name

We want the prices of several tariffs, with insurance for a parcel worth 3000:

```
insuredAll :: [Double] -> [Double]
insuredAll prices = map withInsurance prices
  where
    withInsurance = \p -> insured 3000 p
```

The local function is used once, and its name says little.

A **lambda** is a function without a name. The backslash stands for λ.
In Racket, it was `(lambda (p) (insured 3000 p))`.

A Function Without a Name

We want the prices of several tariffs, with insurance for a parcel worth 3000:

```
insuredAll :: [Double] -> [Double]
insuredAll prices = map (\p -> insured 3000 p) prices
```

The local function is used once, and its name says little.

A **lambda** is a function without a name. The backslash stands for λ.
In Racket, it was `(lambda (p) (insured 3000 p))`.

The name only labelled the lambda. So we pass the lambda itself.

Currying

Now look at `insured` itself:

```
insured :: Double -> Double -> Double
insured value price = price + 0.01 * value
```

Currying

Now look at `insured` itself:

```
insured :: Double -> Double -> Double
insured = \value price -> price + 0.01 * value
```

A lambda may take several arguments.

Currying

Now look at `insured` itself:

```
insured :: Double -> (Double -> Double)
insured = \value -> \price -> price + 0.01 * value
```

A lambda may take several arguments.

A lambda of two arguments is a lambda that returns a lambda.
The arrow groups to the right, so the parentheses in the type change nothing.

Currying

Now look at `insured` itself:

```
insured :: Double -> (Double -> Double)
insured value = \price -> price + 0.01 * value
```

A lambda may take several arguments.

A lambda of two arguments is a lambda that returns a lambda.
The arrow groups to the right, so the parentheses in the type change nothing.

Thus, `insured` takes the value and returns a function of the price.

Currying

Now look at `insured` itself:

```
insured :: Double -> (Double -> Double)
insured value = \price -> price + 0.01 * value
```

A lambda may take several arguments.

A lambda of two arguments is a lambda that returns a lambda.
The arrow groups to the right, so the parentheses in the type change nothing.

Thus, `insured` takes the value and returns a function of the price.

Every function in Haskell takes one argument.
This is **currying**, named after Haskell Curry.

Partial Application

Back to the lambda we passed to `map`:

```
insuredAll prices = map (\p -> insured 3000 p) prices
```

Partial Application

Back to the lambda we passed to `map`:

```
insuredAll prices = map (\p -> (insured 3000) p) prices
```

A call groups to the left, so `insured 3000 p` is `(insured 3000) p`.

Partial Application



Back to the lambda we passed to `map`:

```
insuredAll prices = map (insured 3000) prices
```

A call groups to the left, so `insured 3000 p` is `(insured 3000) p`.

The lambda only passes `p` on to the function `insured 3000`.
So we can pass that function itself.

Partial Application

Back to the lambda we passed to `map`:

```
insuredAll prices = map (insured 3000) prices
```

A call groups to the left, so `insured 3000 p` is `(insured 3000) p`.

The lambda only passes `p` on to the function `insured 3000`.
So we can pass that function itself.

Giving a function fewer arguments than it takes is **partial application**.
The sections of last week are partial applications of operators:

```
filter (\w -> w > 2000) weights
filter (> 2000) weights
```

Partial Application

Back to the lambda we passed to `map`:

```
insuredAll prices = map (insured 3000) prices
```

A call groups to the left, so `insured 3000 p` is `(insured 3000) p`.

The lambda only passes `p` on to the function `insured 3000`.
So we can pass that function itself.

Giving a function fewer arguments than it takes is **partial application**.
The sections of last week are partial applications of operators:

```
filter (\w -> w > 2000) weights
filter (> 2000) weights
```

In Racket, `(insured 3000)` is an error, unless `insured` returns a lambda.

Three Ways to Pass a Function



	local function	lambda	partial application
passed as	`withInsurance`	`\p -> insured 3000 p`	`insured 3000`
reads best for	a step that needs a name	a short step, used once	a missing last argument
Racket	a local **`define`**	**`lambda`**	a returned **`lambda`**

Three Ways to Pass a Function

	local function	lambda	partial application
passed as	`withInsurance`	`\p -> insured 3000 p`	`insured 3000`
reads best for	a step that needs a name	a short step, used once	a missing last argument
Racket	a local **`define`**	**`lambda`**	a returned **`lambda`**

All three give the same function.

Three Ways to Pass a Function

	local function	lambda	partial application
passed as	`withInsurance`	`\p -> insured 3000 p`	`insured 3000`
reads best for	a step that needs a name	a short step, used once	a missing last argument
Racket	a local **`define`**	**`lambda`**	a returned **`lambda`**

All three give the same function.

A name helps when the step is long, or when the name explains it.
A partial application or a section fits only when the argument comes last.

Three Ways to Pass a Function

	local function	lambda	partial application
passed as	`withInsurance`	`\p -> insured 3000 p`	`insured 3000`
reads best for	a step that needs a name	a short step, used once	a missing last argument
Racket	a local **`define`**	**`lambda`**	a returned **`lambda`**

All three give the same function.

A name helps when the step is long, or when the name explains it.
A partial application or a section fits only when the argument comes last.

Choose the one that reads best.

Question: What Is Left?



Recall `countHeavy` from last week:

```
countHeavy :: Double -> [Double] -> Int
```

Question

What are the types of these three expressions?

```
ghci> :type insured 3000

ghci> :type map postage

ghci> :type countHeavy 1000
```

Question: What Is Left?

Recall `countHeavy` from last week:

```
countHeavy :: Double -> [Double] -> Int
```

Question

What are the types of these three expressions?

```
ghci>  :type insured 3000
insured 3000 ::  Double -> Double
ghci>  :type map postage

ghci>  :type countHeavy 1000
```

Question: What Is Left?

Recall `countHeavy` from last week:

```
countHeavy :: Double -> [Double] -> Int
```

Question

What are the types of these three expressions?

```
ghci>  :type insured 3000
insured 3000 ::  Double -> Double
ghci>  :type map postage
map postage ::  [Double] -> [Double]
ghci>  :type countHeavy 1000
```

Question: What Is Left?



Recall `countHeavy` from last week:

```
countHeavy :: Double -> [Double] -> Int
```

Question

What are the types of these three expressions?

```
ghci>  :type insured 3000
insured 3000 ::  Double -> Double
ghci>  :type map postage
map postage ::  [Double] -> [Double]
ghci>  :type countHeavy 1000
countHeavy 1000 ::  [Double] -> Int
```

Question: What Is Left?

Recall `countHeavy` from last week:

```
countHeavy :: Double -> [Double] -> Int
```

Question

What are the types of these three expressions?

```
ghci>  :type insured 3000
insured 3000 ::  Double -> Double
ghci>  :type map postage
map postage ::  [Double] -> [Double]
ghci>  :type countHeavy 1000
countHeavy 1000 ::  [Double] -> Int
```

Each argument we give removes one arrow from the front of the type.

Parametric Polymorphism



```
map :: (a -> b) -> [a] -> [b]
```

```
ghci>  map show weights
["250.0","1200.0","800.0","5400.0","90.0"]
ghci>  map length ["books", "socks"]
[5,5]
```

Parametric Polymorphism



```
map :: (a -> b) -> [a] -> [b]
```

```
ghci>  map show weights
["250.0","1200.0","800.0","5400.0","90.0"]
ghci>  map length ["books", "socks"]
[5,5]
```

`a` and `b` are type variables, and each use of `map` chooses them.
Here they are **`Double`** and **`String`**, then **`String`** and **`Int`**.

Parametric Polymorphism



```
map :: (a -> b) -> [a] -> [b]
```

```
ghci>  map show weights
["250.0","1200.0","800.0","5400.0","90.0"]
ghci>  map length ["books", "socks"]
[5,5]
```

`a` and `b` are type variables, and each use of `map` chooses them.
Here they are **`Double`** and **`String`**, then **`String`** and **`Int`**.

One definition works for every choice.
This is **parametric polymorphism**.

Parametric Polymorphism



```
map :: (a -> b) -> [a] -> [b]
```

```
ghci>  map show weights
["250.0","1200.0","800.0","5400.0","90.0"]
ghci>  map length ["books", "socks"]
[5,5]
```

`a` and `b` are type variables, and each use of `map` chooses them.
Here they are **`Double`** and **`String`**, then **`String`** and **`Int`**.

One definition works for every choice.
This is **parametric polymorphism**.

In `map postage` on the previous slide, both are **`Double`**.

Question: What Can It Do?

```haskell
f :: [a] -> Int
g :: a -> a
```

Question

What can `f` do with the elements? And `g` with its argument?

[1] Philip Wadler (1989). **"Theorems for Free!"** In: *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*. URL: https://doi.org/10.1145/99370.99404.

Question: What Can It Do?

```haskell
f :: [a] -> Int
g :: a -> a
```

Question

What can `f` do with the elements? And `g` with its argument?

`f` knows nothing about `a`.
It cannot compare, add or show the elements. It can only count them.

[1] Philip Wadler (1989). **"Theorems for Free!"** In: *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*. URL: https://doi.org/10.1145/99370.99404.

Question: What Can It Do?

```haskell
f :: [a] -> Int
g :: a -> a
```

Question

What can `f` do with the elements? And `g` with its argument?

`f` knows nothing about `a`.
It cannot compare, add or show the elements. It can only count them.

`g` must return an `a`, and the only one it has is its argument.
So `g x = x` is the only function of this type that always returns.

[1] Philip Wadler (1989). **"Theorems for Free!"** In: *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*. URL: https://doi.org/10.1145/99370.99404.

Question: What Can It Do?

```haskell
f :: [a] -> Int
g :: a -> a
```

Question

What can `f` do with the elements? And `g` with its argument?

`f` knows nothing about `a`.
It cannot compare, add or show the elements. It can only count them.

`g` must return an `a`, and the only one it has is its argument.
So `g x = x` is the only function of this type that always returns.

Thus, a polymorphic type tells us much of what the function does[1].

[1] Philip Wadler (1989). **"Theorems for Free!"** In: *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*. URL: https://doi.org/10.1145/99370.99404.

`flip` and `(.)`

```haskell
flip :: (a -> b -> c) -> b -> a -> c
(.)  :: (b -> c) -> (a -> b) -> a -> c
```

`flip` and `(.)`



```
flip :: (a -> b -> c) -> b -> a -> c
(.)  :: (b -> c) -> (a -> b) -> a -> c
```

`flip` swaps the two arguments of a function:

```
ghci>  map (flip insured 400) [1000, 3000]
[410.0,430.0]
```

`flip` and `(.)`



```
flip :: (a -> b -> c) -> b -> a -> c
(.)  :: (b -> c) -> (a -> b) -> a -> c
```

`flip` swaps the two arguments of a function:

```
ghci>  map (flip insured 400) [1000, 3000]
[410.0,430.0]
```

Here the price stays 400, and the value changes.

`flip` and `(.)`



```
flip :: (a -> b -> c) -> b -> a -> c
(.)  :: (b -> c) -> (a -> b) -> a -> c
```

`flip` swaps the two arguments of a function:

```
ghci>  map (flip insured 400) [1000, 3000]
[410.0,430.0]
```

Here the price stays 400, and the value changes.

`withNds . postage` applies `postage`, then `withNds`:

```
ghci>  map (withNds . postage) weights
[300.0,480.0,480.0,1080.0,300.0]
```

`flip` and `(.)`

```
flip :: (a -> b -> c) -> b -> a -> c
(.)  :: (b -> c) -> (a -> b) -> a -> c
```

`flip` swaps the two arguments of a function:

```
ghci>  map (flip insured 400) [1000, 3000]
[410.0,430.0]
```

Here the price stays 400, and the value changes.

`withNds . postage` applies `postage`, then `withNds`:

```
ghci>  map (withNds . postage) weights
[300.0,480.0,480.0,1080.0,300.0]
```

We read `.` as ∘ in mathematics: the function on the right applies first.

zip and zipWith

```
zip     :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

`zip` and `zipWith`

```
zip     :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

`zip` pairs the elements, and `zipWith` combines them with a function.

zip and zipWith

```
zip     :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

`zip` pairs the elements, and `zipWith` combines them with a function.

```
line :: Int -> String -> String
line i n = show i ++ ". " ++ n
```

```
ghci>  zip [1 ..] ["books", "socks"]
[(1,"books"),(2,"socks")]
ghci>  zipWith line [1 ..] ["books", "socks"]
["1.  books","2.  socks"]
```

zip and zipWith

```
zip     :: [a] -> [b] -> [(a, b)]
zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
```

`zip` pairs the elements, and `zipWith` combines them with a function.

```
line :: Int -> String -> String
line i n = show i ++ ". " ++ n
```

```
ghci>  zip [1 ..] ["books", "socks"]
[(1,"books"),(2,"socks")]
ghci>  zipWith line [1 ..] ["books", "socks"]
["1.  books","2.  socks"]
```

Both stop at the end of the shorter list.

Thus, `[1 ..]` numbers a list of any length.

Folds

An Accumulator, Twice

```haskell
totalWeight :: [Double] -> Double
totalWeight ws = go 0 ws
  where go acc []       = acc
        go acc (x : xs) = go (acc + x) xs

heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where go best []       = best
        go best (x : xs) = go (max best x) xs
```

An Accumulator, Twice

```haskell
totalWeight :: [Double] -> Double
totalWeight ws = go 0 ws
  where go acc []       = acc
        go acc (x : xs) = go (acc + x) xs

heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where go best []       = best
        go best (x : xs) = go (max best x) xs
```

`max` returns the larger of two numbers. What differs between the two?

An Accumulator, Twice

```haskell
totalWeight :: [Double] -> Double
totalWeight ws = go 0 ws
  where go acc []       = acc
        go acc (x : xs) = go (acc + x) xs

heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where go best []       = best
        go best (x : xs) = go (max best x) xs
```

`max` returns the larger of two numbers. What differs between the two?

1. The start: `0`, and the first weight.
2. The update with each element: `acc + x`, and `max best x`.

An Accumulator, Twice

```haskell
totalWeight :: [Double] -> Double
totalWeight ws = go 0 ws
  where go acc []       = acc
        go acc (x : xs) = go (acc + x) xs

heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where go best []       = best
        go best (x : xs) = go (max best x) xs
```

`max` returns the larger of two numbers. What differs between the two?

1. The start: `0`, and the first weight.
2. The update with each element: `acc + x`, and `max best x`.

The rest is the same, so we pass the two differences as arguments.

The Left Fold



```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

The Left Fold

```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

The step takes the accumulator first, then the element[2]:

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

The Left Fold

```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

The step takes the accumulator first, then the element[2]:

```haskell
foldl' (+) 0 [250, 1200]
```

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

The Left Fold

```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

The step takes the accumulator first, then the element[2]:

```haskell
foldl' (+) 0 [250, 1200]
= foldl' (+) 250 [1200]                                    0 + 250
```

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

The Left Fold



```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

The step takes the accumulator first, then the element[2]:

```haskell
foldl' (+) 0 [250, 1200]
= foldl' (+) 250 [1200]                    0 + 250
= foldl' (+) 1450 []                       250 + 1200
```

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

The Left Fold

```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

The step takes the accumulator first, then the element[2]:

```haskell
foldl' (+) 0 [250, 1200]
= foldl' (+) 250 [1200]                    0 + 250
= foldl' (+) 1450 []                       250 + 1200
= 1450
```

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

The Left Fold

```haskell
foldl' :: (b -> a -> b) -> b -> [a] -> b
totalWeight ws     = foldl' (+) 0 ws
heaviest (w : ws) = foldl' max w ws
```

The step takes the accumulator first, then the element[2]:

```haskell
foldl' (+) 0 [250, 1200]
= foldl' (+) 250 [1200]                    0 + 250
= foldl' (+) 1450 []                       250 + 1200
= 1450
```

Two more folds

Haskell also has `foldr` and `foldl`. We compare the three in topic 9.

[2] `foldl'` is in the Prelude from GHC 9.10. Older versions need `import Data.List (foldl')`. GHCi prints its type with `Foldable`, which the last section explains.

Racket's `foldl` and Haskell's `foldl'`



The same idea, and the arguments of the step in the opposite order:

Racket: the element, then the accumulator

```
> (foldl cons empty '(1 2 3))
'(3 2 1)
> (foldl - 10000 '(400 250))
9850
```

Haskell: the accumulator, then the element

```
ghci> foldl' (flip (:)) [] [1, 2, 3]
[3,2,1]
ghci> foldl' (-) 10000 [400, 250]
9350.0
```

Racket's `foldl` and Haskell's `foldl'`



The same idea, and the arguments of the step in the opposite order:

Racket: the element, then the accumulator

```
> (foldl cons empty '(1 2 3))
'(3 2 1)
> (foldl - 10000 '(400 250))
9850
```

Haskell: the accumulator, then the element

```
ghci> foldl' (flip (:)) [] [1, 2, 3]
[3,2,1]
ghci> foldl' (-) 10000 [400, 250]
9350.0
```

With `(:)` in place of `flip (:)`, GHC refuses the program.

Racket's `foldl` and Haskell's `foldl'`

The same idea, and the arguments of the step in the opposite order:

Racket: the element, then the accumulator

```
> (foldl cons empty '(1 2 3))
'(3 2 1)
> (foldl - 10000 '(400 250))
9850
```

Haskell: the accumulator, then the element

```
ghci> foldl' (flip (:)) [] [1, 2, 3]
[3,2,1]
ghci> foldl' (-) 10000 [400, 250]
9350.0
```

With `(:)` in place of `flip (:)`, GHC refuses the program.

With `-`, both languages accept the program, and the answers differ.

Racket's `foldl` and Haskell's `foldl'`



The same idea, and the arguments of the step in the opposite order:

Racket: the element, then the accumulator

```
> (foldl cons empty '(1 2 3))
'(3 2 1)
> (foldl - 10000 '(400 250))
9850
```

Haskell: the accumulator, then the element

```
ghci> foldl' (flip (:)) [] [1, 2, 3]
[3,2,1]
ghci> foldl' (-) 10000 [400, 250]
9350.0
```

With `(:)` in place of `flip (:)`, GHC refuses the program.

With `-`, both languages accept the program, and the answers differ.

When you go back to Racket

There the step takes the element first.

Use the Most Specific Function

The number of heavy parcels, with a left fold:

```haskell
countHeavy :: Double -> [Double] -> Int
countHeavy limit ws =
  foldl' (\n w -> if w > limit then n + 1 else n) 0 ws
```

Use the Most Specific Function

The number of heavy parcels, with a left fold:

```
countHeavy :: Double -> [Double] -> Int
countHeavy limit ws =
  foldl' (\n w -> if w > limit then n + 1 else n) 0 ws
```

`sum` and `length` are left folds with a name.

`filter` and `map` can be written as folds too.

Use the Most Specific Function

The number of heavy parcels, with a left fold:

```
countHeavy :: Double -> [Double] -> Int
countHeavy limit ws =
  foldl' (\n w -> if w > limit then n + 1 else n) 0 ws
```

`sum` and `length` are left folds with a name.
`filter` and `map` can be written as folds too.

When one of them fits, use it:

```
countHeavy limit ws = length (filter (> limit) ws)
```

Use the Most Specific Function

The number of heavy parcels, with a left fold:

```
countHeavy :: Double -> [Double] -> Int
countHeavy limit ws =
  foldl' (\n w -> if w > limit then n + 1 else n) 0 ws
```

`sum` and `length` are left folds with a name.
`filter` and `map` can be written as folds too.

When one of them fits, use it:

```
countHeavy limit ws = length (filter (> limit) ws)
```

It says the same as the fold above, and it is easier to read.

5 min break



Types of Our Own

Four Promises

Last week ended with four places where a type said too little:

1. `discount` took a **`Bool`**. It meant a member or a non-member.
2. `zone` took a **`String`**. It meant one of a few countries.
3. `firstWeight` had no case for **`[]`**, and GHC could only warn.
4. `heaviest []` returned `0`. The result is optional, or the input is never empty.

Four Promises



Last week ended with four places where a type said too little:

1. `discount` took a **`Bool`**. It meant a member or a non-member.
2. `zone` took a **`String`**. It meant one of a few countries.
3. `firstWeight` had no case for **`[]`**, and GHC could only warn.
4. `heaviest` **`[]`** returned `0`. The result is optional, or the input is never empty.

The next slides fix them in this order.

Enumerations



```haskell
data Customer = Member | NonMember

discount :: Customer -> Int -> Double
discount Member n    = if n >= 10 then 0.2 else 0.1
discount NonMember n = if n >= 10 then 0.1 else 0
```

Enumerations

```haskell
data Customer = Member | NonMember

discount :: Customer -> Int -> Double
discount Member n    = if n >= 10 then 0.2 else 0.1
discount NonMember n = if n >= 10 then 0.1 else 0
```

`data` declares a new type and its **constructors**. The bar `|` reads as “or”. The names of types and constructors start with a capital letter.

Enumerations

```haskell
data Customer = Member | NonMember

discount :: Customer -> Int -> Double
discount Member n    = if n >= 10 then 0.2 else 0.1
discount NonMember n = if n >= 10 then 0.1 else 0
```

`data` declares a new type and its **constructors**. The bar `|` reads as “or”. The names of types and constructors start with a capital letter.

What does `discount True 12` give now?

Enumerations



```haskell
data Customer = Member | NonMember

discount :: Customer -> Int -> Double
discount Member n    = if n >= 10 then 0.2 else 0.1
discount NonMember n = if n >= 10 then 0.1 else 0
```

`data` declares a new type and its **constructors**. The bar `|` reads as “or”.
The names of types and constructors start with a capital letter.

What does `discount True 12` give now?
A type error: GHC expected a `Customer`, and found a `Bool`.

Enumerations

```haskell
data Customer = Member | NonMember

discount :: Customer -> Int -> Double
discount Member n    = if n >= 10 then 0.2 else 0.1
discount NonMember n = if n >= 10 then 0.1 else 0
```

`data` declares a new type and its **constructors**. The bar `|` reads as “or”.
The names of types and constructors start with a capital letter.

What does `discount True 12` give now?
A type error: GHC expected a `Customer`, and found a `Bool`.

The Prelude declares `Bool` in the same way:

```haskell
data Bool = False | True
```

Every Case, Checked



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia

zone :: Country -> Int
zone Russia     = 1
zone Kazakhstan = 2
zone Belarus    = 2
```

Every Case, Checked



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia

zone :: Country -> Int
zone Russia     = 1
zone Kazakhstan = 2
zone Belarus    = 2
```

```
Parcels.hs:6:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘zone’:
        Patterns of type ‘Country’ not matched: Armenia
```

GHC knows all four countries, so it names the one we forgot.
With a `String`, it could not know them.

Every Case, Checked



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia

zone :: Country -> Int
zone Russia     = 1
zone Kazakhstan = 2
zone Belarus    = 2
```

```
Parcels.hs:6:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘zone’:
        Patterns of type ‘Country’ not matched: Armenia
```

GHC knows all four countries, so it names the one we forgot.
With a **`String`**, it could not know them.

An equation `zone _ = 3` at the end would hide Armenia, and any new country.

One Record for Three Cases?

A parcel is picked up, brought to an address by a courier, or left in a locker. With what we had last week, we would write:

```
data Method   = Pickup | Courier | Locker
data Delivery = Delivery Method String Int   -- address, locker

courier, locker :: Delivery
courier = Delivery Courier "Universitetskaya 1" 0
locker  = Delivery Locker "" 17
```

One Record for Three Cases?

A parcel is picked up, brought to an address by a courier, or left in a locker.
With what we had last week, we would write:

```haskell
data Method   = Pickup | Courier | Locker
data Delivery = Delivery Method String Int   -- address, locker

courier, locker :: Delivery
courier = Delivery Courier "Universitetskaya 1" 0
locker  = Delivery Locker "" 17
```

What is the address of a locker delivery?
And what is the locker of a courier delivery?

One Record for Three Cases?

A parcel is picked up, brought to an address by a courier, or left in a locker.
With what we had last week, we would write:

```
data Method   = Pickup | Courier | Locker
data Delivery = Delivery Method String Int   -- address, locker

courier, locker :: Delivery
courier = Delivery Courier "Universitetskaya 1" 0
locker  = Delivery Locker "" 17
```

What is the address of a locker delivery?
And what is the locker of a courier delivery?

Neither exists, so the fields hold placeholders: `""` and `0`.
Nothing stops a wrong value either: GHC accepts `Delivery Pickup "" 17`,
a pickup from the depot with a locker.

Constructors With Fields



```haskell
data Delivery = Pickup | Courier String | Locker Int

instructions :: Delivery -> String
instructions Pickup         = "collect at the office"
instructions (Courier addr) = "bring to " ++ addr
instructions (Locker n)     = "leave in locker " ++ show n
```

Constructors With Fields

```haskell
data Delivery = Pickup | Courier String | Locker Int

instructions :: Delivery -> String
instructions Pickup         = "collect at the office"
instructions (Courier addr) = "bring to " ++ addr
instructions (Locker n)     = "leave in locker " ++ show n
```

Each constructor carries the fields of its own case, and no others.

Constructors With Fields

```haskell
data Delivery = Pickup | Courier String | Locker Int

instructions :: Delivery -> String
instructions Pickup         = "collect at the office"
instructions (Courier addr) = "bring to " ++ addr
instructions (Locker n)     = "leave in locker " ++ show n
```

Each constructor carries the fields of its own case, and no others.

A pattern names the fields of its constructor.
So `addr` exists only in the equation for **`Courier`**.

Constructors With Fields

```haskell
data Delivery = Pickup | Courier String | Locker Int

instructions :: Delivery -> String
instructions Pickup         = "collect at the office"
instructions (Courier addr) = "bring to " ++ addr
instructions (Locker n)     = "leave in locker " ++ show n
```

Each constructor carries the fields of its own case, and no others.

A pattern names the fields of its constructor.
So `addr` exists only in the equation for **`Courier`**.

A constructor with fields is a function that builds a value of the type:

```
ghci>  :type Locker
Locker ::  Int -> Delivery
```

Constructors With Fields

```haskell
data Delivery = Pickup | Courier String | Locker Int

instructions :: Delivery -> String
instructions Pickup         = "collect at the office"
instructions (Courier addr) = "bring to " ++ addr
instructions (Locker n)     = "leave in locker " ++ show n
```

Each constructor carries the fields of its own case, and no others.

A pattern names the fields of its constructor.
So `addr` exists only in the equation for **`Courier`**.

A constructor with fields is a function that builds a value of the type:

```
ghci>  :type Locker
Locker ::  Int -> Delivery
```

In Racket, each case was a **`struct`**, and a **`cond`** asked `courier?` and `locker?`.

Records



```haskell
data Parcel = Parcel
  { contents    :: String
  , weight      :: Double
  , destination :: Country }
```

```haskell
books :: Parcel
books = Parcel
  { contents    = "books"
  , weight      = 5400
  , destination = Armenia }
```

Records

```
data Parcel = Parcel
  { contents    :: String
  , weight      :: Double
  , destination :: Country }
```

```
books :: Parcel
books = Parcel
  { contents    = "books"
  , weight      = 5400
  , destination = Armenia }
```

A **record** gives the fields names.
Each name is also a function that reads its field.

Records

```haskell
data Parcel = Parcel
  { contents    :: String
  , weight      :: Double
  , destination :: Country }
```

```haskell
books :: Parcel
books = Parcel
  { contents    = "books"
  , weight      = 5400
  , destination = Armenia }
```

A **record** gives the fields names.

Each name is also a function that reads its field.

```
ghci> weight (books { weight = 5600 })
5600.0
ghci> weight books
5400.0
```

An update makes a new parcel, and `books` stays as it was.

In Racket, **`struct`** gave `parcel-weight`, and **`struct-copy`** the copy.

A Type That Contains Itself



```haskell
data Expr = Lit Int | Add Expr Expr | Mul Expr Expr

eval :: Expr -> Int
eval (Lit n)   = n
eval (Add a b) = eval a + eval b
eval (Mul a b) = eval a * eval b
```

```
ghci>  eval (Add (Lit 2) (Mul (Lit 3) (Lit 4)))
14
```

A Type That Contains Itself



```haskell
data Expr = Lit Int | Add Expr Expr | Mul Expr Expr

eval :: Expr -> Int
eval (Lit n)   = n
eval (Add a b) = eval a + eval b
eval (Mul a b) = eval a * eval b
```

```
ghci>  eval (Add (Lit 2) (Mul (Lit 3) (Lit 4)))
14
```

An expression is a literal, or a sum or a product of two expressions.
`eval` has one equation per constructor, and a recursive call per **`Expr`** in it.

A Type That Contains Itself



```haskell
data Expr = Lit Int | Add Expr Expr | Mul Expr Expr

eval :: Expr -> Int
eval (Lit n)   = n
eval (Add a b) = eval a + eval b
eval (Mul a b) = eval a * eval b
```

```
ghci>  eval (Add (Lit 2) (Mul (Lit 3) (Lit 4)))
14
```

An expression is a literal, or a sum or a product of two expressions.
`eval` has one equation per constructor, and a recursive call per **`Expr`** in it.

`Add` and **`Mul`** contain the type twice, so an expression is a tree:
above, the syntax tree that a parser builds for `2 + 3 * 4`.

A Type That Contains Itself



```haskell
data Expr = Lit Int | Add Expr Expr | Mul Expr Expr

eval :: Expr -> Int
eval (Lit n)   = n
eval (Add a b) = eval a + eval b
eval (Mul a b) = eval a * eval b
```

```
ghci>  eval (Add (Lit 2) (Mul (Lit 3) (Lit 4)))
14
```

An expression is a literal, or a sum or a product of two expressions.
`eval` has one equation per constructor, and a recursive call per **`Expr`** in it.

`Add` and **`Mul`** contain the type twice, so an expression is a tree:
above, the syntax tree that a parser builds for `2 + 3 * 4`.

A list contains itself once: empty, or an element in front of a list.

`Maybe` and `Either`

Maybe: A Result That May Be Missing



```haskell
data Maybe a = Nothing | Just a                    -- in the Prelude

heaviest :: [Double] -> Maybe Double
heaviest []       = Nothing
heaviest (w : ws) = Just (foldl' max w ws)

firstWeight :: [Double] -> Maybe Double
firstWeight []      = Nothing
firstWeight (w : _) = Just w
```

Maybe: A Result That May Be Missing



```haskell
data Maybe a = Nothing | Just a                    -- in the Prelude

heaviest :: [Double] -> Maybe Double
heaviest []       = Nothing
heaviest (w : ws) = Just (foldl' max w ws)

firstWeight :: [Double] -> Maybe Double
firstWeight []      = Nothing
firstWeight (w : _) = Just w
```

`Maybe` has a **type parameter** `a`, as a list has one for its elements.

`Maybe Double` and `Maybe Country` are two different types.

Maybe: A Result That May Be Missing

```haskell
data Maybe a = Nothing | Just a                    -- in the Prelude

heaviest :: [Double] -> Maybe Double
heaviest []       = Nothing
heaviest (w : ws) = Just (foldl' max w ws)

firstWeight :: [Double] -> Maybe Double
firstWeight []      = Nothing
firstWeight (w : _) = Just w
```

`Maybe` has a **type parameter** `a`, as a list has one for its elements.

`Maybe Double` and `Maybe Country` are two different types.

The empty list now gets `Nothing` rather than a guess.
The type tells every caller to handle `Nothing`.

Using a `Maybe`

```haskell
describeHeaviest :: [Double] -> String
describeHeaviest ws =
  case heaviest ws of
    Nothing -> "no parcels"
    Just w  -> "the heaviest is " ++ show w
```

Using a Maybe



```haskell
describeHeaviest :: [Double] -> String
describeHeaviest ws =
  case heaviest ws of
    Nothing -> "no parcels"
    Just w  -> "the heaviest is " ++ show w
```

The function `maybe` does the same in one call, with a default for `Nothing` and a function for what is inside `Just`:

```haskell
maybe :: b -> (a -> b) -> Maybe a -> b
```

```
ghci>  maybe 0 postage (heaviest weights)
900.0
ghci>  maybe 0 postage (heaviest [])
0.0
```

`NonEmpty`: An Input That Is Never Empty



The other fix promised last week: ask for a list that cannot be empty.

```haskell
import Data.List.NonEmpty (NonEmpty (..), nonEmpty)
-- data NonEmpty a = a :| [a]
heaviest :: NonEmpty Double -> Double
heaviest (w :| ws) = foldl' max w ws
```

`NonEmpty`: An Input That Is Never Empty



The other fix promised last week: ask for a list that cannot be empty.

```haskell
import Data.List.NonEmpty (NonEmpty (..), nonEmpty)
-- data NonEmpty a = a :| [a]
heaviest :: NonEmpty Double -> Double
heaviest (w :| ws) = foldl' max w ws
```

A `NonEmpty` has a first element and a list for the rest.

So `heaviest` needs no case for an empty list.

`NonEmpty`: An Input That Is Never Empty



The other fix promised last week: ask for a list that cannot be empty.

```haskell
import Data.List.NonEmpty (NonEmpty (..), nonEmpty)
-- data NonEmpty a = a :| [a]
heaviest :: NonEmpty Double -> Double
heaviest (w :| ws) = foldl' max w ws
```

A `NonEmpty` has a first element and a list for the rest.
So `heaviest` needs no case for an empty list.

`nonEmpty` does the check where the list comes from:

```
ghci> nonEmpty [250, 1200]
Just (250 :| [1200])
ghci> nonEmpty []
Nothing
```

`NonEmpty`: An Input That Is Never Empty



The other fix promised last week: ask for a list that cannot be empty.

```haskell
import Data.List.NonEmpty (NonEmpty (..), nonEmpty)
-- data NonEmpty a = a :| [a]
heaviest :: NonEmpty Double -> Double
heaviest (w :| ws) = foldl' max w ws
```

A `NonEmpty` has a first element and a list for the rest.
So `heaviest` needs no case for an empty list.

`nonEmpty` does the check where the list comes from:

```
ghci>  nonEmpty [250, 1200]
Just (250 :| [1200])
ghci>  nonEmpty []
Nothing
```

`Maybe` says the answer may be missing, and `NonEmpty` that the input never is.

`Either`: A Result or a Reason



```haskell
data Either a b = Left a | Right b              -- in the Prelude

checkWeight :: Double -> Either String Double
checkWeight w
  | w <= 0     = Left "the weight must be positive"
  | w > 30000 = Left "too heavy for a courier"
  | otherwise = Right w
```

`Either`: A Result or a Reason



```haskell
data Either a b = Left a | Right b              -- in the Prelude

checkWeight :: Double -> Either String Double
checkWeight w
  | w <= 0     = Left "the weight must be positive"
  | w > 30000 = Left "too heavy for a courier"
  | otherwise = Right w
```

By convention, `Left` holds the failure, and `Right` holds the result.

`Either`: A Result or a Reason



```haskell
data Either a b = Left a | Right b              -- in the Prelude

checkWeight :: Double -> Either String Double
checkWeight w
  | w <= 0     = Left "the weight must be positive"
  | w > 30000  = Left "too heavy for a courier"
  | otherwise  = Right w
```

By convention, `Left` holds the failure, and `Right` holds the result.

`Nothing` only says that something failed, and `Left` also says what.

`Either`: A Result or a Reason



```haskell
data Either a b = Left a | Right b              -- in the Prelude

checkWeight :: Double -> Either String Double
checkWeight w
  | w <= 0     = Left "the weight must be positive"
  | w > 30000  = Left "too heavy for a courier"
  | otherwise  = Right w
```

By convention, `Left` holds the failure, and `Right` holds the result.

`Nothing` only says that something failed, and `Left` also says what.

In Racket we returned `#f`, or raised an error.
Here the failure is part of the type, so the caller has to handle it.

Counting Values

How many values does each type have?

1. `Customer`
2. `Country`
3. `(Customer, Country)`
4. `Either Customer Country`
5. `Maybe Country`
6. `Country -> Customer`

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Counting Values

How many values does each type have?

1. `Customer` — 2
2. `Country` — 4
3. `(Customer, Country)`
4. `Either Customer Country`
5. `Maybe Country`
6. `Country -> Customer`

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Counting Values

How many values does each type have?

1. `Customer` — 2
2. `Country` — 4
3. (`Customer`, `Country`) — $2 \times 4 = 8$
4. `Either Customer Country`
5. `Maybe Country`
6. `Country -> Customer`

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Counting Values

How many values does each type have?

1. `Customer` — 2
2. `Country` — 4
3. (`Customer`, `Country`) — $2 \times 4 = 8$
4. `Either Customer Country` — $2 + 4 = 6$
5. `Maybe Country`
6. `Country -> Customer`

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Counting Values

How many values does each type have?

1. `Customer` — 2
2. `Country` — 4
3. (`Customer`, `Country`) — $2 \times 4 = 8$
4. `Either Customer Country` — $2 + 4 = 6$
5. `Maybe Country` — $1 + 4 = 5$
6. `Country -> Customer`

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Counting Values

How many values does each type have?

1. `Customer` — 2
2. `Country` — 4
3. (`Customer`, `Country`) — $2 \times 4 = 8$
4. `Either Customer Country` — $2 + 4 = 6$
5. `Maybe Country` — $1 + 4 = 5$
6. `Country -> Customer` — $2^4 = 16$: a customer for each of the four countries

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Counting Values

How many values does each type have?

1. `Customer` — 2
2. `Country` — 4
3. (`Customer`, `Country`) — $2 \times 4 = 8$
4. `Either Customer Country` — $2 + 4 = 6$
5. `Maybe Country` — $1 + 4 = 5$
6. `Country -> Customer` — $2^4 = 16$: a customer for each of the four countries

The counts of the constructors add up, and the counts of the fields multiply. This is the idea behind the name **algebraic data types**[3].

[3] Joel Burget (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types.

Task: A Status

A parcel is at the depot, in transit to a country, or delivered on a date.

a first attempt, with `import Data.Time (Day)`

```
data Status = Status
  { delivered   :: Bool
  , inTransitTo :: Maybe Country
  , deliveredOn :: Maybe Day }
```

Hint

Which fields does each of the three situations need?

Three minutes, on paper

1. Write a value of this type that describes no real parcel.
2. Declare a type `Status` that has no such values.
3. Write `isDelivered :: Status -> Bool` for your type.

Task: A Status, Solved



`Status True (Just Armenia) Nothing` is delivered and in transit, with no date.

[4] Yaron Minsky (Mar. 2011). ***Effective ML Revisited.*** Jane Street Tech Blog. URL: https://blog.janestreet.com/effective-ml-revisited/.

Task: A Status, Solved

`Status True (Just Armenia) Nothing` is delivered and in transit, with no date.

```
data Status = AtDepot | InTransit Country | Delivered Day

isDelivered :: Status -> Bool
isDelivered (Delivered _) = True
isDelivered _             = False
```

Each situation is a constructor with the fields it needs, and no others.

[4] Yaron Minsky (Mar. 2011). ***Effective ML Revisited.*** Jane Street Tech Blog. URL: https://blog.janestreet.com/effective-ml-revisited/.

Task: A Status, Solved

`Status True (Just Armenia) Nothing` is delivered and in transit, with no date.

```
data Status = AtDepot | InTransit Country | Delivered Day

isDelivered :: Status -> Bool
isDelivered (Delivered _) = True
isDelivered _             = False
```

Each situation is a constructor with the fields it needs, and no others.

Here `_` is fine: every status other than a delivery gives `False`, now and later.

[4] Yaron Minsky (Mar. 2011). ***Effective ML Revisited.*** Jane Street Tech Blog. URL: https://blog.janestreet.com/effective-ml-revisited/.

Task: A Status, Solved

`Status True (Just Armenia) Nothing` is delivered and in transit, with no date.

```
data Status = AtDepot | InTransit Country | Delivered Day

isDelivered :: Status -> Bool
isDelivered (Delivered _) = True
isDelivered _             = False
```

Each situation is a constructor with the fields it needs, and no others.

Here `_` is fine: every status other than a delivery gives `False`, now and later.

We looked for states that make no sense, and left them out of the type.
The slogan is “make illegal states unrepresentable”[4].

[4] Yaron Minsky (Mar. 2011). ***Effective ML Revisited.*** Jane Street Tech Blog. URL: https://blog.janestreet.com/effective-ml-revisited/.

Type Classes

What => Means



```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

```
ghci> lookup "socks" parcels
Just 90.0
ghci> lookup "piano" parcels
Nothing
```

[5] Philip Wadler and Stephen Blott (1989). **"How to Make Ad-Hoc Polymorphism Less Ad Hoc".** In: *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/75277.75283.

What => Means



```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

```
ghci> lookup "socks" parcels
Just 90.0
ghci> lookup "piano" parcels
Nothing
```

`lookup` compares keys with `==`, and `Eq a` asks the key type to provide it.

[5] Philip Wadler and Stephen Blott (1989). **"How to Make Ad-Hoc Polymorphism Less Ad Hoc".** In: *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/75277.75283.

What => Means

```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

```
ghci> lookup "socks" parcels
Just 90.0
ghci> lookup "piano" parcels
Nothing
```

`lookup` compares keys with `==`, and `Eq a` asks the key type to provide it.

A **type class** is a group of operations that a type may provide[5]:

```
class Eq a where (==), (/=) :: a -> a -> Bool
```

[5] Philip Wadler and Stephen Blott (1989). **"How to Make Ad-Hoc Polymorphism Less Ad Hoc".** In: *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/75277.75283.

What => Means

```
lookup :: Eq a => a -> [(a, b)] -> Maybe b
```

```
ghci> lookup "socks" parcels
Just 90.0
ghci> lookup "piano" parcels
Nothing
```

`lookup` compares keys with `==`, and `Eq a` asks the key type to provide it.

A **type class** is a group of operations that a type may provide[5]:

```
class Eq a where (==), (/=) :: a -> a -> Bool
```

An **instance** says how one type provides them.

[5] Philip Wadler and Stephen Blott (1989). **"How to Make Ad-Hoc Polymorphism Less Ad Hoc".** In: *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/75277.75283.

Question: A Lookup by Country

```haskell
tariffs :: [(Country, Double)]
tariffs = [(Russia, 1.0), (Armenia, 2.0)]
```

Question

Does `lookup Armenia tariffs` compile?

Question: A Lookup by Country

```
tariffs :: [(Country, Double)]
tariffs = [(Russia, 1.0), (Armenia, 2.0)]
```

Question

Does `lookup Armenia tariffs` compile?

```
Parcels.hs:9:11: error: [GHC-39999]
    • No instance for ‘Eq Country’ arising from a use of ‘lookup’
```

No. A type of our own provides no operations until we give it instances.

Instances for Free: `deriving`

```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

Instances for Free: `deriving`

```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

- `Show` writes a value as the code that makes it, so GHCi can print it.

Instances for Free: `deriving`

```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

- `Show` writes a value as the code that makes it, so GHCi can print it.
- `Eq` compares the constructors, then the fields.

Instances for Free: `deriving`

```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

- `Show` writes a value as the code that makes it, so GHCi can print it.
- `Eq` compares the constructors, then the fields.
- `Ord` orders the constructors as they are declared, then the fields.

Instances for Free: `deriving`



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

- `Show` writes a value as the code that makes it, so GHCi can print it.
- `Eq` compares the constructors, then the fields.
- `Ord` orders the constructors as they are declared, then the fields.

```
ghci> lookup Armenia tariffs
Just 2.0
```

Instances for Free: `deriving`



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

- `Show` writes a value as the code that makes it, so GHCi can print it.
- `Eq` compares the constructors, then the fields.
- `Ord` orders the constructors as they are declared, then the fields.

```
ghci> lookup Armenia tariffs
Just 2.0
ghci> maximum [Belarus, Russia]
```

Instances for Free: `deriving`



```haskell
data Country = Russia | Kazakhstan | Belarus | Armenia
  deriving (Show, Eq, Ord)
```

- `Show` writes a value as the code that makes it, so GHCi can print it.
- `Eq` compares the constructors, then the fields.
- `Ord` orders the constructors as they are declared, then the fields.

```
ghci> lookup Armenia tariffs
Just 2.0
ghci> maximum [Belarus, Russia]
Belarus
```

`Belarus` is declared after `Russia`, so it is the greater one.

An Instance by Hand



A tariff zone is a set of countries, which we keep as a list:

```
newtype Zone = Zone [Country] deriving Eq
```

```
ghci> Zone [Russia, Belarus] == Zone [Belarus, Russia]
```

An Instance by Hand

A tariff zone is a set of countries, which we keep as a list:

```
newtype Zone = Zone [Country] deriving Eq
```

```
ghci> Zone [Russia, Belarus] == Zone [Belarus, Russia]
False
```

The derived `==` compares the lists in order.

So we drop `deriving Eq`, and write our own instance:

An Instance by Hand

A tariff zone is a set of countries, which we keep as a list:

```
newtype Zone = Zone [Country] deriving Eq
```

```
ghci>  Zone [Russia, Belarus] == Zone [Belarus, Russia]
False
```

The derived `==` compares the lists in order.

So we drop `deriving Eq`, and write our own instance:

```
instance Eq Zone where
  Zone xs == Zone ys = all (`elem` ys) xs && all (`elem` xs) ys

all  :: (a -> Bool) -> [a] -> Bool      -- holds for every element?
elem :: Eq a => a -> [a] -> Bool        -- is in the list?
```

An Instance by Hand

A tariff zone is a set of countries, which we keep as a list:

```
newtype Zone = Zone [Country] deriving Eq
```

```
ghci>  Zone [Russia, Belarus] == Zone [Belarus, Russia]
False
```

The derived `==` compares the lists in order.
So we drop `deriving Eq`, and write our own instance:

```
instance Eq Zone where
  Zone xs == Zone ys = all (`elem` ys) xs && all (`elem` xs) ys

all  :: (a -> Bool) -> [a] -> Bool      -- holds for every element?
elem :: Eq a => a -> [a] -> Bool        -- is in the list?
```

Two zones are now equal when each country of one is in the other.

The Constraint the Body Asks For

```
heavierThan limit xs = filter (> limit) xs
```

The Constraint the Body Asks For

```
heavierThan limit xs = filter (> limit) xs
```

Nothing here says **`Double`**. What is the type?

The Constraint the Body Asks For

```
heavierThan limit xs = filter (> limit) xs
```

Nothing here says **`Double`**. What is the type?

```
heavierThan :: Ord a => a -> [a] -> [a]
```

The body uses `>`, and `>` belongs to the class **`Ord`**.

The Constraint the Body Asks For

```
heavierThan limit xs = filter (> limit) xs
```

Nothing here says **`Double`**. What is the type?

```
heavierThan :: Ord a => a -> [a] -> [a]
```

The body uses `>`, and `>` belongs to the class **`Ord`**.

If we leave the constraint out of the signature, GHC asks for it:

```
Parcels.hs:4:32: error: [GHC-39999]
    • No instance for ‘Ord a’ arising from a use of ‘>’
```

One Class Includes Another

```haskell
class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>), (>=) :: a -> a -> Bool
```

One Class Includes Another

```haskell
class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>), (>=) :: a -> a -> Bool
```

Every instance of `Ord` must also be an instance of `Eq`.

`Eq` is a **superclass** of `Ord`.

One Class Includes Another

```haskell
class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>), (>=) :: a -> a -> Bool
```

Every instance of **`Ord`** must also be an instance of **`Eq`**.

`Eq` is a **superclass** of **`Ord`**.

Which constraint here is redundant?

```haskell
between :: (Eq a, Ord a) => a -> a -> a -> Bool
between lo hi x = lo <= x && x <= hi
```

One Class Includes Another

```haskell
class Eq a => Ord a where
  compare :: a -> a -> Ordering
  (<), (<=), (>), (>=) :: a -> a -> Bool
```

Every instance of `Ord` must also be an instance of `Eq`.
`Eq` is a **superclass** of `Ord`.

Which constraint here is redundant?

```haskell
between :: (Eq a, Ord a) => a -> a -> a -> Bool
between lo hi x = lo <= x && x <= hi
```

`Eq a`, since `Ord a` already includes it.
`-Wall` does not report it, and `-Wredundant-constraints` does.

Reading `Foldable` in a Signature

GHCi prints the list functions of this lecture with a more general type:

```
ghci>  :type sum
sum ::  (Foldable t, Num a) => t a -> a
```

Reading `Foldable` in a Signature



GHCi prints the list functions of this lecture with a more general type:

```
ghci>  :type sum
sum ::  (Foldable t, Num a) => t a -> a
```

`t` is a container type, such as a list or `Maybe`, and for a list `t a` is `[a]`.
Read `Foldable t => t a` as “a list, or another container that can be folded”:

```
ghci>  sum (250 :| [1200])
1450
ghci>  length (Just 3)
1
```

Reading `Foldable` in a Signature



GHCi prints the list functions of this lecture with a more general type:

```
ghci>  :type sum
sum ::  (Foldable t, Num a) => t a -> a
```

`t` is a container type, such as a list or `Maybe`, and for a list `t a` is `[a]`.
Read `Foldable t => t a` as “a list, or another container that can be folded”:

```
ghci>  sum (250 :| [1200])
1450
ghci>  length (Just 3)
1
```

Some functions of topic 8 say `Traversable t`: containers that can be rebuilt.

The Most General Type

```
pairWith f x = (x, f x)
```

[6] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/582153.582176.

The Most General Type

```
pairWith f x = (x, f x)
```

By hand, as GHC does it[6]:

1. Give each unknown a fresh variable: `x :: a`.

[6] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/582153.582176.

The Most General Type

```
pairWith f x = (x, f x)
```

By hand, as GHC does it[6]:

1. Give each unknown a fresh variable: `x :: a`.
2. `f` is applied to `x`, so `f :: a -> b` for some `b`.

[6] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/582153.582176.

The Most General Type

```
pairWith f x = (x, f x)
```

By hand, as GHC does it[6]:

1. Give each unknown a fresh variable: `x :: a`.
2. `f` is applied to `x`, so `f :: a -> b` for some `b`.
3. The result is the pair `(a, b)`.

[6] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/582153.582176.

The Most General Type

```
pairWith f x = (x, f x)
```

By hand, as GHC does it[6]:

1. Give each unknown a fresh variable: `x :: a`.
2. `f` is applied to `x`, so `f :: a -> b` for some `b`.
3. The result is the pair `(a, b)`.

```
pairWith :: (a -> b) -> a -> (a, b)
```

Any other type of `pairWith` is this one with `a` and `b` replaced.

[6] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/582153.582176.

The Most General Type

```
pairWith f x = (x, f x)
```

By hand, as GHC does it[6]:

1. Give each unknown a fresh variable: `x :: a`.
2. `f` is applied to `x`, so `f :: a -> b` for some `b`.
3. The result is the pair `(a, b)`.

```
pairWith :: (a -> b) -> a -> (a, b)
```

Any other type of `pairWith` is this one with `a` and `b` replaced.

Not everything has a type.
In `\f -> f f`, from Ω of topic 1, `f` would need the types `a` and `a -> b` at once.

[6] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. URL: https://doi.org/10.1145/582153.582176.

What We Can Read Now



	Racket	Haskell
a function without a name	`(lambda (w) ...)`	`\w -> ...`
a function with one argument fixed	a returned `lambda`	`insured 3000`
a fold with an accumulator	`foldl`, the element first	`foldl'`, the accumulator first
one of several cases	`struct`s and `cond`	several constructors
a record, and a changed copy	`struct`, `struct-copy`	`p { weight = 5600 }`
no answer	`#f`, or an error	`Maybe`, or `Either`

What We Can Read Now

	Racket	Haskell
a function without a name	`(lambda (w) ...)`	`\w -> ...`
a function with one argument fixed	a returned `lambda`	`insured 3000`
a fold with an accumulator	`foldl`, the element first	`foldl'`, the accumulator first
one of several cases	`struct`s and `cond`	several constructors
a record, and a changed copy	`struct`, `struct-copy`	`p { weight = 5600 }`
no answer	`#f`, or an error	`Maybe`, or `Either`

And two things we have not seen in Racket:

- type variables, as in the type of `map`;
- constraints, as in `Eq a =>`.

Quick Recap

1. What is the type of `insured 3000`?
2. Which argument comes first in the step of `foldl'`? And in Racket's `foldl`?
3. What can a function of type `[a] -> Int` do with the elements?
4. When is `Either` better than `Maybe`?
5. What does `Ord a =>` in a signature ask for?

Quick Recap

1. What is the type of `insured 3000`?
   `Double -> Double`: one argument given, one arrow gone.
2. Which argument comes first in the step of `foldl'`? And in Racket's `foldl`?
3. What can a function of type `[a] -> Int` do with the elements?
4. When is **`Either`** better than **`Maybe`**?
5. What does **`Ord a =>`** in a signature ask for?

Quick Recap

1. What is the type of `insured 3000`?
   `Double -> Double`: one argument given, one arrow gone.
2. Which argument comes first in the step of `foldl'`? And in Racket's `foldl`?
   The accumulator in Haskell, and the element in Racket.
3. What can a function of type `[a] -> Int` do with the elements?
4. When is **`Either`** better than **`Maybe`**?
5. What does **`Ord`** `a =>` in a signature ask for?

Quick Recap

1. What is the type of `insured 3000`?
   `Double -> Double`: one argument given, one arrow gone.
2. Which argument comes first in the step of `foldl'`? And in Racket's `foldl`?
   The accumulator in Haskell, and the element in Racket.
3. What can a function of type `[a] -> Int` do with the elements?
   Nothing but count them, since it knows nothing about `a`.
4. When is **`Either`** better than **`Maybe`**?
5. What does **`Ord`** `a =>` in a signature ask for?

Quick Recap

1. What is the type of `insured 3000`?
   `Double -> Double`: one argument given, one arrow gone.
2. Which argument comes first in the step of `foldl'`? And in Racket's `foldl`?
   The accumulator in Haskell, and the element in Racket.
3. What can a function of type `[a] -> Int` do with the elements?
   Nothing but count them, since it knows nothing about `a`.
4. When is **`Either`** better than **`Maybe`**?
   When the caller needs to know why there is no result.
5. What does **`Ord`** `a =>` in a signature ask for?

Quick Recap

1. What is the type of `insured 3000`?
   `Double -> Double`: one argument given, one arrow gone.
2. Which argument comes first in the step of `foldl'`? And in Racket's `foldl`?
   The accumulator in Haskell, and the element in Racket.
3. What can a function of type `[a] -> Int` do with the elements?
   Nothing but count them, since it knows nothing about `a`.
4. When is **`Either`** better than **`Maybe`**?
   When the caller needs to know why there is no result.
5. What does **`Ord`** `a =>` in a signature ask for?
   That the type `a` provide comparisons, and therefore also `==`.

Quiz for Lecture 7

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 7

Ten minutes, on Moodle, now.
Seven questions on this lecture, then three about the lecture itself.

No AI. The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

Say what you learned and what stayed unclear, **specifically**.
I read them before writing the next lecture.

What's Next

In today's lab you will

- write the worksheet on this lecture
- defend mini project 1 or problem set 6, if your name is drawn
- start on problem set 7

→ **Before next week: input and output**

- Read *Learn You a Haskell*, Chapters 8 and 9 (Lipovača 2011): our own types and type classes, input and output.
- Load today's module in GHCi, delete an equation of `zone`, and read the warning GHC gives.

Thank you!



References

- Burget, Joel (June 2015). ***The Algebra (and Calculus!) of Algebraic Data Types.*** Code Words, Issue Three. Recurse Center. URL: https://codewords.recurse.com/issues/three/algebra-and-calculus-of-algebraic-data-types (visited on 10/01/2026).
- Damas, Luis and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '82. Albuquerque, New Mexico: Association for Computing Machinery, pp. 207–212. ISBN: 0897910656. DOI: 10.1145/582153.582176. URL: https://doi.org/10.1145/582153.582176.

References ii

- Lipovača, Miran (2011). ***Learn You a Haskell for Great Good! A Beginner's Guide.*** No Starch Press. URL: https://learnyouahaskell.com/chapters (visited on 09/27/2026).
- Marlow, Simon (2010). ***Haskell 2010 Language Report.*** URL: https://www.haskell.org/onlinereport/haskell2010/ (visited on 09/27/2026).
- Minsky, Yaron (Mar. 2011). ***Effective ML Revisited.*** Jane Street Tech Blog. URL: https://blog.janestreet.com/effective-ml-revisited/ (visited on 10/01/2026).

References iii

- The GHC Team (2024). ***GHC User's Guide, version 9.10***. URL: https://downloads.haskell.org/ghc/9.10.3/docs/users_guide/ (visited on 09/27/2026).
- Wadler, Philip (1989). **"Theorems for Free!"** In: *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*. FPCA '89. Association for Computing Machinery, pp. 347–359. DOI: 10.1145/99370.99404. URL: https://doi.org/10.1145/99370.99404.

References iv



Wadler, Philip and Stephen Blott (1989). **“How to Make Ad-Hoc Polymorphism Less Ad Hoc”.** In: *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’89. Association for Computing Machinery, pp. 60–76. DOI: 10.1145/75277.75283. URL: https://doi.org/10.1145/75277.75283.